

AURA: Project Overview & Interview Cheat Sheet

Autonomous Unified Research Assistant — High-Level Technical Summary

1. Project Overview & Purpose

AURA (Autonomous Unified Research Assistant) is a full-stack, production-grade multi-agent AI system built to execute domain-specific autonomous reasoning across web research, document QA, code generation, dataset profiling, and visual OCR inspection.

Core Problems Solved:

- **Context Contamination:** Replaces monolithic prompts with specialized domain agents operating under targeted system instructions.
- **Vector Leakage:** Enforces metadata-scoped vector retrieval in ChromaDB, preventing cross-document contamination.
- **Knowledge Cutoffs:** Fuses live Tavily web research with vector retrieval for real-time comparative synthesis.
- **Interactive Data Profiling:** Automated Pandas tabular profiling paired with dynamic React visual bar charts.

2. Quick Interview Elevator Pitches

30-Second Summary:

"AURA is a multi-agent AI platform built with Python, FastAPI, LangChain, Mistral AI, ChromaDB, and React. Instead of relying on a single prompt, an Auto-Router dynamically classifies user queries and delegates execution across 6 specialized agents—including Web Research, Document RAG, Code Engineering, Pandas Profiling, and Vision OCR. It features strict vector metadata scoping and cross-session memory."

1-Minute Summary:

"Most AI wrappers suffer from context bloat and vector leakage. I built AURA to solve this using a hub-and-spoke multi-agent architecture. The FastAPI backend receives requests, and an Intent Classifier routes queries to specialized LCEL pipelines. For document QA, AURA indexes PDFs into ChromaDB using Mistral embeddings and retrieves context via MMR search with metadata filters. For tabular datasets, it runs Pandas profiling and outputs structured JSON rendered into interactive visual bar charts by React. The system is deployed on Render and Vercel Edge CDN."

3. Tech Stack & Selection Rationale

Technology	Role	Why Chosen (Selection Rationale)
Python 3.10+	Backend Language	Rich AI ecosystem, native LangChain support, async loop support.
FastAPI	API Gateway	ASGI non-blocking execution, high concurrency, Pydantic validation.
LangChain (LCEL)	Pipeline Orchestration	Declarative pipeline composition via pipe operator (), async support.
Mistral AI	Core LLM & Embeddings	Superior instruction-following, competitive reasoning, 1024-dim dense embeddings.
Pixtral 12B	Vision-Language Model	Visual OCR and optical inspection over schematics and diagrams.
ChromaDB	Vector Database	Embedded disk persistence (HNSW index), zero external server cluster dependency.
Tavily Search API	Real-Time Search	Purpose-built for LLMs, returns clean extracted markdown text.
BeautifulSoup4	Web Scraper	Lightweight HTML text parsing for deep comparative research.
React 18 + Vite	Frontend Framework	Fast Virtual DOM, fast HMR build tooling, responsive component state.
TailwindCSS	UI Styling Engine	Utility-first modern dark design system and responsive grid breakpoints.

4. End-to-End Project Workflow

- Step 1: User Request** → User submits query/file via React 18 UI.
- Step 2: API Ingestion** → FastAPI validates request payloads via Pydantic (`/api/chat`).
- Step 3: Intent Routing** → Auto-Router classifies intent (``rag``, ``research``, ``code``, ``data``, ``vision``, ``chat``).
- Step 4: Pipeline Execution** → Invokes target LCEL chain binding prompts, external tools, or ChromaDB.
- Step 5: Memory Sync** → Appends interaction history and user facts into ``storage/db.json``.
- Step 6: UI Rendering** → Returns JSON response; React renders markdown text and interactive bar charts.

5. Summary of 6 Specialized Agents

1. **Auto Router Agent** (``auto``): Classifies query intent and manages stateful global user memory.
2. **Deep Research Agent** (``research``): Multi-step Tavily search & BeautifulSoup scraper generating comparative reports.
3. **RAG Document QA Agent** (``rag``): PDF/DOCX chunking, Mistral embeddings, and ChromaDB MMR search with metadata filters.
4. **Code Engineer Agent** (``code``): Generates clean, fundamental code with $O(1)/O(N)$ complexity metrics.
5. **Data Analysis Agent** (``data``): Profiles CSV datasets via Pandas and outputs visual JSON bar charts.
6. **Vision & OCR Agent** (``vision``): Processes image pixels via ``pixtral-12b-2409`` for visual schematic inspection.

6. Top 15 Essential Interview Q&A;

Q1: What is the core problem AURA solves?

AURA eliminates context clutter, tool hallucinations, and vector cross-leakage by routing requests across 6 domain-focused agents instead of a single monolithic prompt.

Q2: How does the Intent Router work?

An intent classifier directive inspects user queries and maps them to target agent keys (``rag``, ``research``, ``code``, ``data``, ``vision``, ``chat``) in ~200ms.

Q3: How do you prevent document cross-contamination in RAG?

By applying strict metadata filtering (``filter={source: file_path}``) during ChromaDB similarity searches so queries only retrieve chunks from the active document.

Q4: What is Maximal Marginal Relevance (MMR) search?

MMR optimizes both query similarity and context diversity, preventing redundant chunks from filling the prompt window.

Q5: Why choose FastAPI over Flask or Django?

FastAPI runs on ASGI (``asyncio``), handling non-blocking network I/O for concurrent LLM requests efficiently while providing Pydantic payload validation.

Q6: Why LCEL over legacy LangChain chains?

LCEL uses a declarative pipe interface (``|``), providing type safety, built-in async support, and explicit component composition without hidden dictionary mechanics.

Q7: What chunking parameters do you use?

1,000-character chunks with a 200-character overlap using ``RecursiveCharacterTextSplitter`` to preserve paragraph context across chunk boundaries.

Q8: How does the Data Agent render visual graphs?

Pandas profiles dataset statistics, the LLM returns a structured JSON chart block, and the React frontend parses this block to render dynamic visual bar charts.

Q9: Why use ChromaDB?

ChromaDB is a lightweight embedded vector store that persists to disk (``storage/chroma_db``) without requiring an external database cluster during development.

Q10: How does AURA handle user session state?

A lightweight JSON database (``storage/db.json``) persists chat history and user facts across server restarts.

Q11: How do you prevent blocking calls in FastAPI?

By running network requests and vector searches inside asynchronous event loops using Python's ``asyncio`` framework.

Q12: How does the Vision Agent process images?

Image files are base64 encoded and dispatched to the ``pixtral-12b-2409`` multimodal model along with structured inspection directives.

Q13: How do you handle UI markdown rendering bugs?

A frontend text sanitizer (``formatMarkdownContent``) uses regex to fix single-line markdown pipe table joins before rendering.

Q14: How would you scale AURA for 100,000 users?

Containerize FastAPI backend pods on AWS ECS, replace embedded ChromaDB with Qdrant/Pinecone, and migrate ``db.json`` to Redis Cloud.

Q15: What is the benefit of setting temperature to 0.0?

It ensures deterministic, reproducible LLM outputs for code generation, data profiling, and structured retrieval.